

Multi-Agent Academic Operating System (MAAOS)

Project Record — Modules, UML Diagrams & Table Design

1. Modules Identified

Module	Core Responsibility
Coordinator Agent	Central orchestrator; routes tasks between all other agents
Email Intelligence Agent	Connects via Gmail API, parses academic emails using LLMs
Priority Agent	Ranks extracted tasks by urgency and importance
Workload Agent	Assesses student's current task load and capacity
Scheduling Agent	Builds personalized study/task schedules
Reminder Agent	Sends timely notifications for deadlines
OCR Agent	Extracts text from uploaded documents/images (Tesseract)
Knowledge (RAG) Agent	Answers questions from notes using ChromaDB retrieval
Analytics Agent	Tracks academic progress and generates insights
Hybrid LLM Router	Switches between cloud/local (Ollama) models for reliability
Auth Module	JWT-based user authentication

2. Module-wise Features

Email Intelligence Agent

- Gmail API polling
- LLM-based entity extraction (assignment, exam, deadline)

Priority Agent

- Deadline-proximity scoring
- Weightage-based ranking of tasks

Workload Agent

- Current task count vs. available study hours
- Overload detection

Scheduling Agent

- Generates day-wise study plan
- Adjusts plan on new task arrival

Reminder Agent

- Push/email notification before deadlines
- Configurable reminder intervals

OCR Agent

- Tesseract-based text extraction from scanned notes/PDFs

RAG Agent

- ChromaDB vector search

- LLM answer generation over uploaded notes

Analytics Agent

- Completion rate tracking
- On-time submission tracking

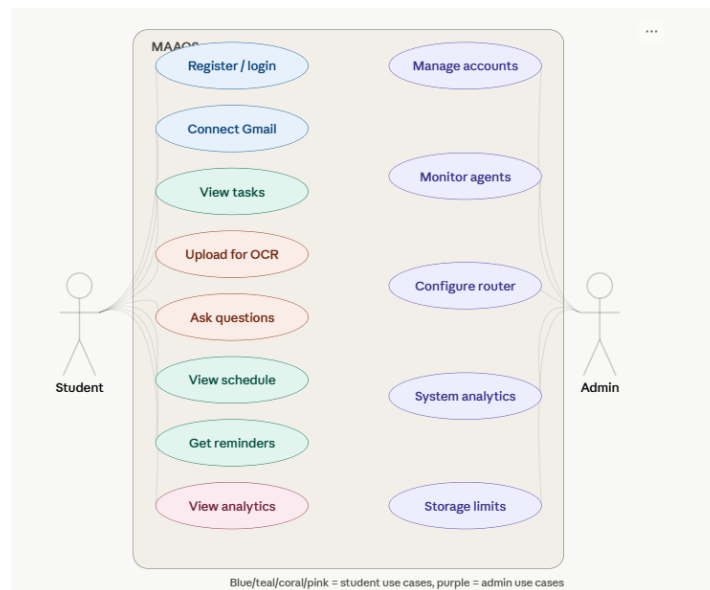
Hybrid LLM Router

- LiteLLM-based switch between cloud API and local Ollama model
- Ensures offline resilience

3. UML & Architecture Diagrams

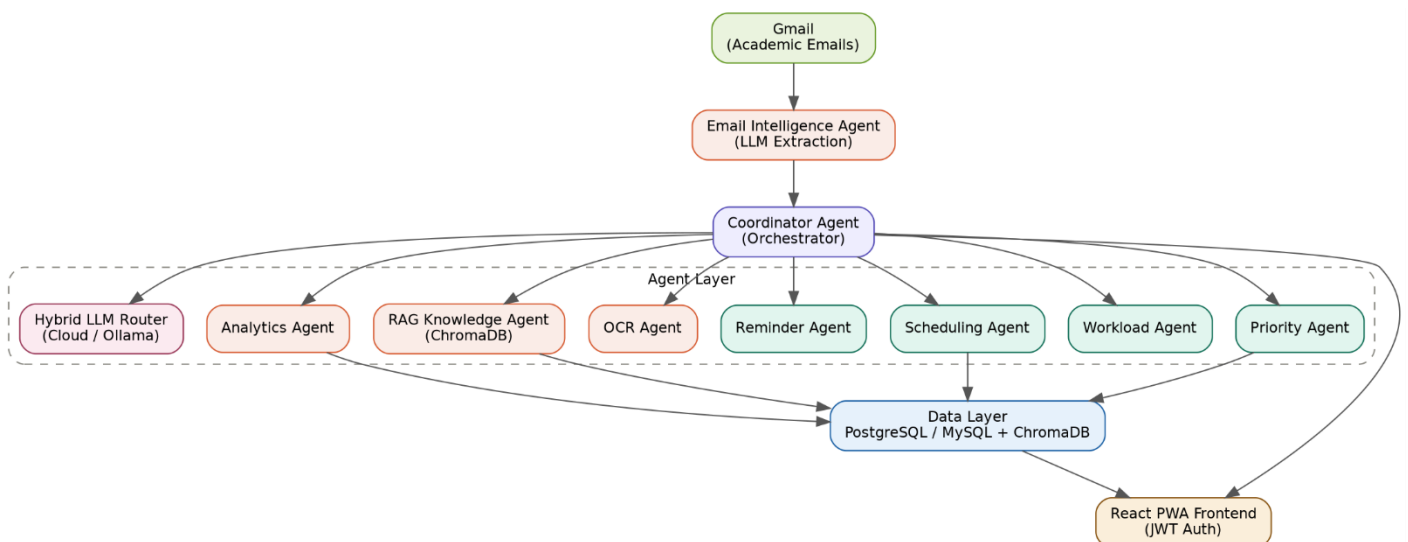
3.1 Use Case Diagram

Depicts the student actor interacting with the primary use cases exposed by MAAOS.



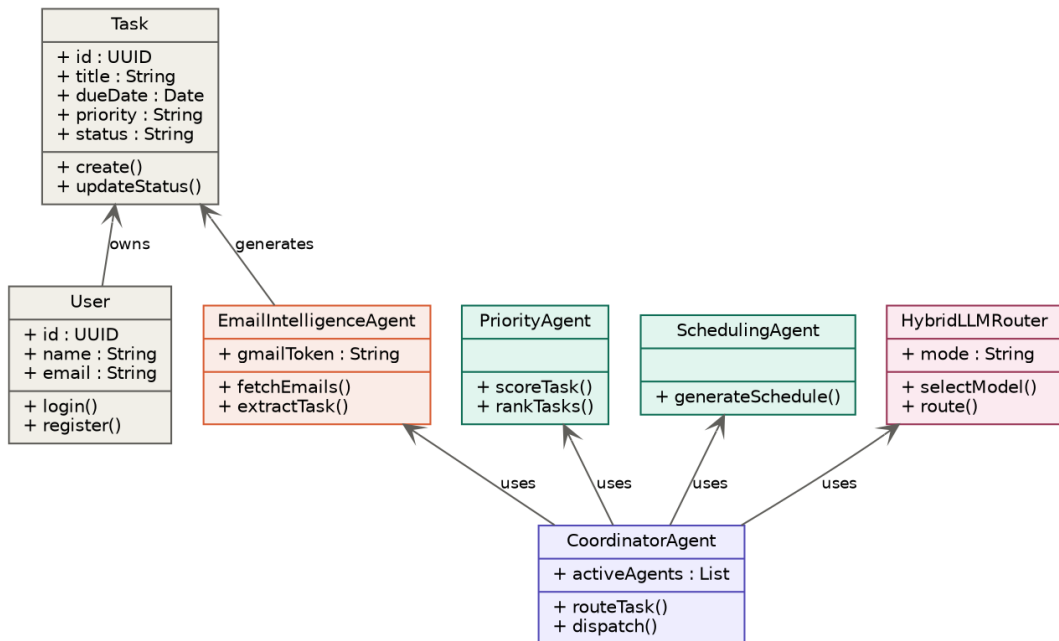
3.2 System Architecture Diagram

Shows the high-level flow from Gmail through the Email Intelligence Agent, the Coordinator Agent, the multi-agent layer, the data layer, and finally the PWA frontend.



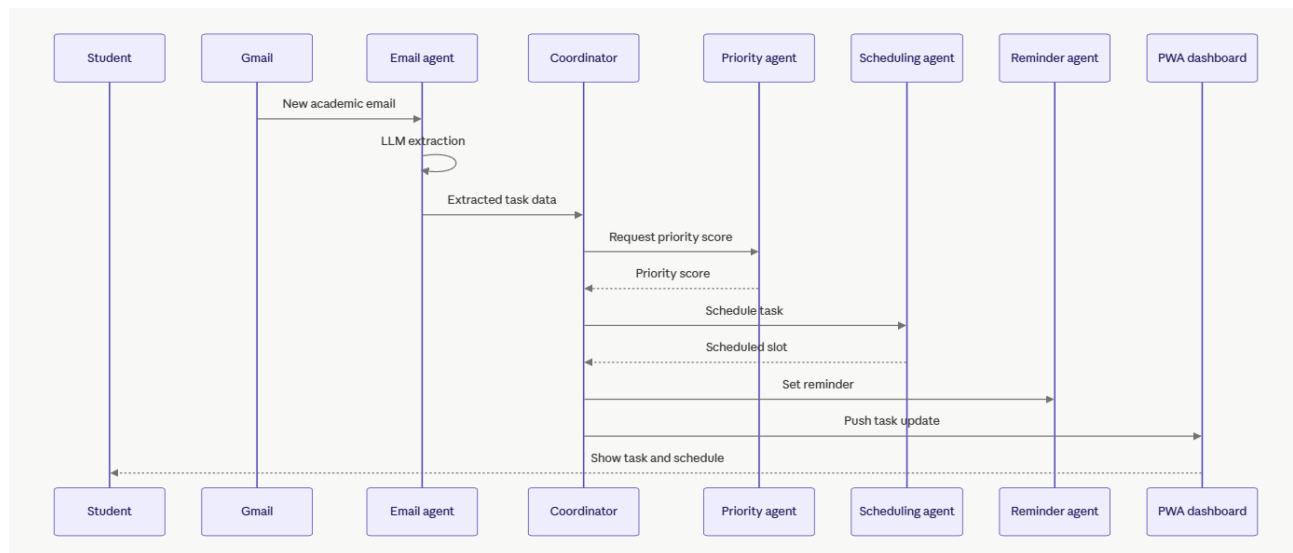
3.3 Class Diagram (Simplified)

Illustrates key classes — User, Task, and the primary agents — with their core attributes and methods, and the associations between them.



3.4 Sequence Diagram — Email to Task Flow

Traces a single academic email as it moves through the system and becomes a scheduled, reminder-backed task.



4. Database / Table Design

The relational schema (PostgreSQL/MySQL) below supports task management, while KNOWLEDGE_CHUNKS stores vector embeddings in ChromaDB for the RAG Agent.

USERS

Field	Type / Constraint
id	UUID (PK)
name	String
email	String
password_hash	String
created_at	Timestamp

EMAILS

Field	Type / Constraint
id	UUID (PK)
user_id	UUID (FK -> USERS)
subject	String
sender	String
body_snippet	Text
received_at	Timestamp

TASKS

Field	Type / Constraint
id	UUID (PK)
user_id	UUID (FK -> USERS)
email_id	UUID (FK -> EMAILS)
title	String
type	String
due_date	Date
priority	String
status	String

REMINDERS

Field	Type / Constraint
id	UUID (PK)
task_id	UUID (FK -> TASKS)
remind_at	Timestamp
sent	Boolean

SCHEDULES

Field	Type / Constraint
id	UUID (PK)
task_id	UUID (FK -> TASKS)
scheduled_date	Date
time_slot	String

DOCUMENTS

Field	Type / Constraint
id	UUID (PK)
user_id	UUID (FK -> USERS)
filename	String
file_type	String
extracted_text	Text

KNOWLEDGE_CHUNKS

Field	Type / Constraint
id	UUID (PK)
document_id	UUID (FK -> DOCUMENTS)
chunk_text	Text
embedding	Vector

ANALYTICS_LOGS

Field	Type / Constraint
id	UUID (PK)
user_id	UUID (FK -> USERS)
metric	String
value	Float
logged_at	Timestamp

Entity Relationships

- USERS 1—N EMAILS (a user receives many emails)
- USERS 1—N TASKS (a user owns many tasks)
- EMAILS 1—N TASKS (an email can generate one or more tasks)

- TASKS 1—N REMINDERS
- TASKS 1—N SCHEDULES
- USERS 1—N DOCUMENTS
- DOCUMENTS 1—N KNOWLEDGE_CHUNKS
- USERS 1—N ANALYTICS_LOGS